

无线电干扰源的快速自动定位与清除模型

摘要

【待填充：开头总述——一句话交代背景与要解决的问题，一句话说明本文总体上采用什么模型与算法体系，并串起四问“单次定位 → 检测点布置 → 多源清除 → 定向源辨识”。】

针对问题一，基于示向度误差有界的特点，建立有界区间角楔交会模型，把每个检测点的示向度按误差界张成角楔，干扰源必在各角楔交会的公共部分之中，形成的定位区域为凸多边形，其直径归结为顶点集上的最远点对。使用程序模拟的五组算例直径在 10 ~ 26 m 之间，而以该直径为直径的圆并不总能覆盖定位区域，五组实验中有两组失败，最远的越界顶点超出半直径约 10.6%。进一步模拟的大量随机算例表明：2 ~ 6 个检测点的失败率约 1.2% ~ 3.3%，最坏时该圆需再放大约 3% 才能覆盖区域。

针对问题二，【待填充：第二个检测点的选择策略与候选区域。】

针对问题三，【待填充：多源定位与清除策略及实测指标。】

针对问题四，【待填充：含定向源的检测与清除策略及实测结果。】

【待填充：结尾总评——四个模型的优点、缺点各一两条，并给出可继续改进的方向。】

关键词：示向度交会 角楔 凸多边形定位区域 直径圆覆盖

一、问题背景与重述

1.1 问题背景

随着无线电技术的普及和广泛应用，频谱管理中的干扰源定位与清除问题日益突出。干扰不仅影响正常通信，还可能危及公共安全。传统清除方式通常受环境遮挡、低功率发射等影响，精准定位耗时长，且危险环境下人工作业风险高，因此研发机器狗自动定位清除系统代替人工执行任务具有现实需求。但任务中通常面临着目标区域范围较大，干扰源数量、频道、位置、有效接收半径及类型未知，测向存在误差，移动、检测、切换频道和清除过程存在耗时，定向干扰源的发射方向无法辨识等困难。如何在信息不完全且时间受限的条件下高效完成任务，是该自动定位清除系统亟待解决的问题。

1.2 问题重述

考虑到上述面临的困难，本文需依次解决以下四个问题：

1. 已知若干检测点的位置及干扰源关于这些检测点的示向度，给出用交会定位法计算可能位置区域（多边形）直径的算法，并回答以该直径为直径的圆能否覆盖此区域。
2. 已知在某一个检测点处测得的示向度，给出第二个检测点的选择策略，并指出该点可取在什么范围（候选区域）之内。
3. 区域内存在多个干扰源，它们占用互不相同的频道且总个数未知，机器狗从给定起点出发自动定位并逐个清除。要求给出策略与算法，经模拟器演练检验后给出清除比例与平均定位清除时间，并据以完成正式测试。
4. 区域内除全向干扰源外还混有定向干扰源，其个数与发射方向均未知，其余条件与问题三相同。要求给出相应的检测与清除策略及算法。

二、问题分析

四个子问题层层递进。问题一是单次交会定位的几何基础；问题二则将该几何模型进一步用作检测点位置的选择依据，形成单源定位策略；问题三在严格的时间预算下，把单源定位能力嵌入“测向、收敛、清除”的在线决策；问题四基于问题三模型的基础之上，再额外引入定向源方向诊断对决策的影响。

2.1 问题一的分析

本问不宜假设误差分布，而应利用题目给出的误差界，采用最坏情形模型：每个检测点的示向度对应一个由“测量方向 $\pm 1^\circ$ ”张成的角楔。多个检测点的角楔求交，即得干扰源的可能定位区域。由于角楔是凸集，其交仍为凸多边形，因此区域直径可转化为凸多边形顶点间的最远点对问题。求解时，先判断交会区域是否有界，再检验以该直径为直径的圆能否覆盖定位区域。整体求解思路可概括为：由误差界构造单点角楔，通过多检测点角楔交会确定凸多边形定位区域，进而将区域直径计算归结为顶点间的最远点对问题；同时辅以有界性判定与圆覆盖检验。

2.2 问题二的分析

【待填充：确定该准则并据此给出选择策略与候选区域。】

2.3 问题三的分析

【待填充：给出策略与算法，并补充模拟与实测指标。】

2.4 问题四的分析

【待填充：给出定向源的检测与清除策略、诊断规则及实测结果。】

三、模型假设

为便于建模求解，在合理范围内作如下假设：

1. 示向度误差有界， $|\Delta\theta_i| \leq \varepsilon$ （本题 $\varepsilon = 1^\circ$ ），且同一检测点的误差在观测期间固定不变，只有更换地点才呈现统计规律；故采用有界区间模型（最坏情形），所得区域保证包含真值。
2. 检测点坐标与示向度读数精确已知；示向度以 x 轴正向逆时针为正，取值于 $[0^\circ, 360^\circ)$ 。
3. 干扰源位置在观测与清除期间固定不变，各源频道互不相同且不随时间变化。
4. 测向机与干扰源位于同一平面，区域内无障碍遮挡。
5. 全向源的覆盖角度为 360° ，定向源为定向方向两侧各 90° （含边界），覆盖范围内信号均匀辐射、场强仅随距离衰减。
6. 机器狗沿直线以 5 m/s 匀速行进，忽略转向与加减速耗时；移动中不可检测，单次检测 5 s ，任意两频道切换 1 s 。
7. 与干扰源距离不超过 20 m 时，光学定位（ 3 s ）与激光清除（ 2 s ）必定成功且与覆盖角度无关；距离不超过 5 m 且处于覆盖角度内时无法获得示向度，可直接定位并清除。

四、符号说明

本文使用的主要符号见表 1，未列出的符号在首次出现处说明。

表 1 主要符号说明

符号	含义	单位
S_i, θ_i	第 i 个检测点的坐标与示向度	$\text{m}, ^\circ$
ε	示向度误差界（本题 1° ）	$^\circ$
$\mathcal{R}, V$	定位区域及其顶点集	—
D	定位区域直径	m
M	直径圆圆心（直径中点）	m
A, B	直径端点	m

五、模型建立与求解

5.1 问题一：示向度交会定位模型

5.1.1 模型建立

设第 i 个检测点为 $S_i = (x_i, y_i)$ ，测得的示向度为 θ_i (x 轴正向逆时针，单位度)，误差界为 $\varepsilon = 1^\circ$ 。由于误差有界，无论真实误差取何值，干扰源都落在以 S_i 为顶点、关于示向度方向对称张开 2ε 的角楔之内：

$$\mathcal{W}_i = \{X : |\beta(S_i, X) - \theta_i| \leq \varepsilon\}, \quad (1)$$

其中 $\beta(S, X)$ 为自 S 指向 X 的方位角。 m 次测向给出的定位区域即各角楔之交。角楔是凸集，也是两个半平面的交；各角楔对干扰源的约束同时成立，故定位区域为 $\mathcal{R} = \bigcap_{i=1}^m \mathcal{W}_i$ ，它是 $2m$ 个半平面之交，必为凸多边形，其顶点数不超过 $2m$ ，且只能落在 $2m$ 条边界直线的交点上。记该区域的顶点集为 V 。

凸多边形上任意两点的最大距离必由两个顶点达到，故求直径不必在连续区域上作最优化，只需在顶点集 V 上比较点对距离：

$$D = \max_{P, Q \in \mathcal{R}} \|P - Q\| = \max_{P, Q \in V} \|P - Q\|. \quad (2)$$

设使式 (2) 取到最大值的两个顶点为 A, B ，则直径圆圆心为 $M = (A + B)/2$ 、半径为 $r = D/2$ 。

若各示向度方向过于接近， $\mathcal{R}$ 沿公共方向无限延伸，式 (2) 无定义，故须先判定有界性。由于无界是无穷远处的性质，判据只涉及方向： $\mathcal{R}$ 无界当且仅当各方位角区间 $\Theta_i = [\theta_i - \varepsilon, \theta_i + \varepsilon]$ 存在公共交集，当 $m = 2$ 时即两次测向的方向夹角不超过 2ε 。该判据只需 $O(m)$ 次角度比较，且不依赖检测点坐标。

覆盖与否由圆盘的凸性直接判定： $\mathcal{R} \subseteq$ 该直径圆当且仅当其全部顶点含于圆内，即

$$\max_{P \in V} \|P - M\| \leq D/2. \quad (3)$$

条件不满足时，必有顶点 P 使 $\angle APB < 90^\circ$ 而落在圆外。注意该圆并把圆心固定在 M 上，与圆心可自由选取的最小包围圆不同，故式 (3) 需逐例检验而不能直接断言成立。

5.1.2 模型求解算法

算法输入为 m 组“检测点坐标与示向度”及误差界 ε ，输出为顶点集 V 、直径 D 、直径端点 A, B 、圆心 M 及覆盖判定，流程如图 1，分四步。

第一步有界性预判：对 m 个方位角区间 $[\theta_i - \varepsilon, \theta_i + \varepsilon]$ 求交（跨 0° 按圆周区间处理），交集非空即区域无界，直接报“不可用”。该步只作角度比较、不需坐标，置于最前可避免在发散区域上继续计算。

第二步区域构造：依次用式 (1) 的 $2m$ 条边界所对应的半平面裁剪一个大矩形，每裁剪一次顶点数至多增加 1，故中间结果不超过 $2m$ 个顶点，裁剪结束即得 $\mathcal{R}$ 。

第三步求直径：取凸包以剔除内部交点与共线中间点，再按式 (2) 枚举顶点对取距离最大者作为 D ，对应两点为 A, B ，圆心 $M = (A + B)/2$ 。枚举量至多 $O(m^2)$ ，本题 $m \leq 20$ ，亦可改用旋转卡壳在 $O(n)$ 内求解。

第四步覆盖判定：按式 (3) 计算 $\max_{P \in V} \|P - M\|$ 并与 $D/2$ 比较，容差随坐标量级取相对量（本文取 $\tau = 10^{-9} \max(1, \max_i \|S_i\|_\infty)$ ），否则相切构型会被舍入误差判为不覆盖；若判定不覆盖，同时输出使该值最大的顶点作为越界位置。

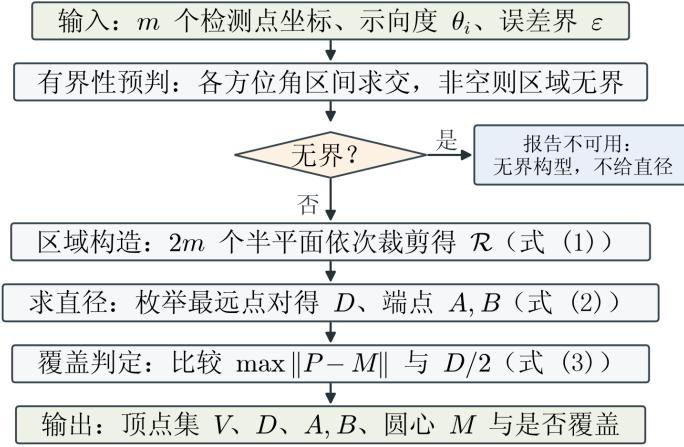


图 1 问题一求解算法流程：无界构型在第 1 步即被拦截，有界构型依次经过区域构造、求直径与覆盖判定

5.1.3 求解结果与分析

取五组自造良态算例（检测点分布在干扰源四周，方位张角 $180^\circ \sim 250^\circ$ 、最小方位间隔不小于 25° 、与干扰源距离 $160 \sim 1300$ m），结果见表 2；五组算例的定位区域直径均在 $10 \sim 26$ m 之间，即两次以上交会已能把候选范围从测向距离的百米量级收缩到数十米以内。表中同时给出覆盖判定，其两种结果的对照见图 2。

表 2 问题一算例的计算结果

算例	m	顶点数	D (m)	$D/2$ (m)	$\max \ P - M\ $ (m)	比值	覆盖
A_1	6	6	17.64	8.82	8.82	1.0000	是
A_2	5	5	18.61	9.31	10.29	1.1055	否
A_3	4	5	25.64	12.82	13.75	1.0722	否
A_4	5	4	21.12	10.56	10.56	1.0000	是
A_5	6	5	10.59	5.30	5.30	1.0000	是

可见五组算例中有两组（ A_2 、 A_3 ）的直径圆无法覆盖定位区域，越界顶点到 M 的距离分别比 $D/2$ 大 0.98 m 与 0.93 m，超出 10.55% 与 7.22%；其原因在于这两组算例的区域是细长的斜四边形，直径为其一条对角线，而越界顶点“斜看”直径的张角小于 90° （ A_2 中该顶点处的张角为 84.2° ），因而落在圆外。

为说明这种失败是否偶然，本文以固定随机种子作两组扫描：在目标区域内随机生成 20000 组两测点构型（只保留有界者），覆盖失败率为 1.16%、最坏比值 1.0336；随机生成 4000 组 3 ~ 6 个检测点的良态构型，其中 3472 个有界构型有 113 个失败、失败率 3.25%、最坏比值 1.0304，即最坏时直径圆需再放大约 3% 才能覆盖区域。

综上，以定位区域直径为直径的圆不一定能覆盖该区域，须按式 (3) 逐例判定；覆

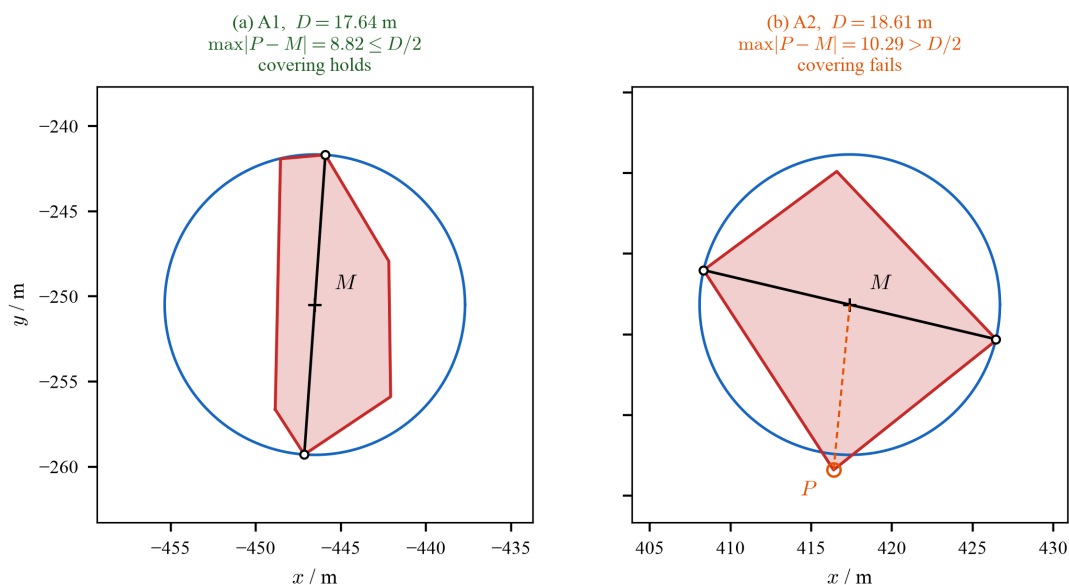


图2 直径圆覆盖判定的两种结果：(a) 覆盖成立（算例 A_1 ，比值 1.0000）；(b) 覆盖不成立（算例 A_2 ，比值 1.1055），橙色虚线标出落在圆外的顶点 P 及其 $|PM| = 10.29 \text{ m} > D/2 = 9.31 \text{ m}$

盖成立时该圆恰为区域的最小包围圆，不成立时其所需半径比 $D/2$ 略大（算例 A_2 、 A_3 分别大约 1.005 倍与 1.001 倍），最小包围圆与该圆的差别由越界顶点给出。

5.2 问题二：第二检测点的选择模型

5.2.1 模型建立

【待填充：给出该问的模型与求解方法。】

5.2.2 求解与结果

【待填充：给出该问的求解结果与分析。】

5.3 问题三：多源定位与清除策略

5.3.1 模型建立

【待填充：给出该问的模型与求解方法。】

5.3.2 求解与结果

【待填充：给出该问的求解结果与实测指标。】

5.4 问题四：含定向源时的定位与清除模型

5.4.1 模型建立

【待填充：给出该问的模型与求解方法。】

5.4.2 求解与结果

【待填充：给出该问的求解结果与分析。】

六、模型评价与改进

6.1 灵敏度分析

【待填充：对模型中的关键参数作灵敏度分析。】

6.2 模型优点

【待填充：完成问题二至四后统一总结四个模型的优点。】

6.3 模型缺点

【待填充：同上，统一总结缺点。】

6.4 改进方向

【待填充：统一给出改进方向。】

参考文献

- [1] 作者. 无线电测向与定位技术 [M]. 出版社, 年份.
- [2] 作者. 论文题目 [J]. 期刊名, 年份, 卷 (期): 页码.
- [3] 全国大学生数学建模竞赛组委会. 全国大学生数学建模竞赛论文格式规范（2026 年修订稿）.

附录 A 支撑材料文件列表

本论文支撑材料所包含的文件见表 3（对应压缩包内的相对路径），问题一的全部算例、结果与插图均由附表中的脚本运行生成，不依赖第三方库与网络。

附录 B 问题一主要源程序

问题一的完整源程序（支撑材料 code/p1_locator.py）见下；其中凸包采用 Andrew 单调链，仅列出与正文式 (1)–(3) 直接对应的部分。

Listing 1: 问题一：交会定位区域直径与直径圆覆盖判定（Python 3）

```
import math
EPS, BOX = 1.0, 1.0e7 # 示向度误差界；裁剪初始包围盒

def rot(v, deg):
    a = math.radians(deg)
    return (v[0] * math.cos(a) - v[1] * math.sin(a),
            v[0] * math.sin(a) + v[1] * math.cos(a))

def dist(p, q):
    return math.hypot(p[0] - q[0], p[1] - q[1])

def bounded(svds, eps=EPS):
    """角楔交集有界 <=> 有向方位角区间公共交集为空。"""
    segs = [(max(0.0, (svds[0] - eps) % 360.0), min(360.0, (svds[0] + eps) % 360.0))]
    for th in svds[1:]:
        lo, hi = (th - eps) % 360.0, (th + eps) % 360.0
```

表 3 支撑材料文件列表（示例均位于 B_locator/ 目录下）

序号	文件路径	说明
1	code/p1_locator.py	问题一源程序（附录 B）
2	code/main.py	问题三、四主程序：模拟器交互与搜索清除
3	Q1/src/p1_solution.py	问题一求解与稳健性扫描（本文数值来源）
4	Q1/figures/ 下三个脚本	问题一插图与算法流程图生成脚本
5	data/p1_case01~05.csv 等	问题一算例数据与真值（真值仅自检用）
6	data/p1_summary.json	问题一求解结果与稳健性统计汇总
7	figures/p1-coverage-contrast.png	直径圆覆盖判定对照（图 2）
8	figures/p1-*.png	角楔交会与直径测量、定位区域与直径圆、算法流程图（图 1）、无界情形示意图

```

parts = [(lo, hi)] if lo <= hi else [(lo, 360.0), (0.0, hi)]
new = []
for a, b in segs:
    for c, d in parts:
        if max(a, c) <= min(b, d) + 1e-12:
            new.append((max(a, c), min(b, d)))
segs = new
if not segs:
    return True
return False

def clip(poly, n, b):
    """保留半平面 {x : n.x + b >= 0}。"""
    out, k = [], len(poly)
    for i in range(k):
        p, q = poly[i], poly[(i + 1) % k]
        fp = n[0] * p[0] + n[1] * p[1] + b
        fq = n[0] * q[0] + n[1] * q[1] + b
        if fp >= 0.0:
            out.append(p)
        if (fp < 0.0 < fq) or (fp > 0.0 > fq):
            t = fp / (fp - fq)
            out.append((p[0] + t * (q[0] - p[0]), p[1] + t * (q[1] - p[1])))
    return out

def region(dets, eps=EPS):
    """定位区域 = 全部角楔之交。dets = [(x, y, theta), ...]"""
    poly = [(-BOX, -BOX), (BOX, -BOX), (BOX, BOX), (-BOX, BOX)]
    for (x, y, th) in dets:
        for s in (-1.0, 1.0):

```

```

        a = math.radians(th + s * eps)
        r = (math.cos(a), math.sin(a)) # 边界射线方向
        n = rot(r, -90.0 * s) # 指向楔内的法向量
        poly = clip(poly, n, -(n[0] * x + n[1] * y))
        if not poly:
            return []
    return poly

def cross(o, a, b):
    return (a[0] - o[0]) * (b[1] - o[1]) - (a[1] - o[1]) * (b[0] - o[0])

def hull(pts):
    """Andrew 单调链（逆时针，弹出共线中间点）。"""
    pts = sorted(set(pts))
    if len(pts) <= 2:
        return pts
    lo, hi = [], []
    for p in pts:
        while len(lo) >= 2 and cross(lo[-2], lo[-1], p) <= 0:
            lo.pop()
        lo.append(p)
    for p in reversed(pts):
        while len(hi) >= 2 and cross(hi[-2], hi[-1], p) <= 0:
            hi.pop()
        hi.append(p)
    return lo[:-1] + hi[::-1]

def solve(dets, eps=EPS):
    """返回直径、直径端点、圆心与覆盖结论。"""
    if not bounded([d[2] for d in dets], eps):
        return {"status": "unbounded"}
    V = hull(region(dets, eps))
    if len(V) < 3:
        return {"status": "degenerate", "vertices": V}
    best, (A, B) = -1.0, (V[0], V[0])
    for i in range(len(V)):
        for j in range(i + 1, len(V)):
            if dist(V[i], V[j]) > best:
                best, (A, B) = dist(V[i], V[j]), (V[i], V[j])
    M = ((A[0] + B[0]) / 2.0, (A[1] + B[1]) / 2.0) # 直径圆圆心
    mx = max(dist(M, p) for p in V)
    scale = max(1.0, max(max(abs(x), abs(y)) for (x, y, _) in dets))
    return {"status": "bounded", "vertices": V, "diameter": best,
            "endpoints": (A, B), "center": M, "radius": best / 2.0,
            "max_vertex_dist": mx,
            "coverage": mx <= best / 2.0 + 1e-9 * scale}

if __name__ == "__main__": # 以算例 A1 自检
    dets = [(-432.855, -596.368, 91.86), (-290.614, -501.995, 121.82),
            (474.860, -160.806, 186.02), (461.755, 349.174, 212.80),
            (-444.749, -62.142, 269.79), (-958.142, 736.087, 298.12)]
    r = solve(dets)
    print("D = %.4f m, max|P-M| = %.4f m, covers = %s"
          % (r["diameter"], r["max_vertex_dist"], r["coverage"]))

```

附录 C 问题三、四主程序（待补）

问题三、问题四的策略与算法完成后，其完整可运行源程序将补充于此（对应支撑材料 `code/main.py`）；该程序负责与模拟器交互，完成多源搜索、定位与清除的全过程。